

Inference in Bayesian Networks

Enumeration, variable elimination, and weighted samples

Learning objectives

By the end of this lesson, you should be able to:

- 01 — Define query, evidence, and hidden variables.
- 02 — Compute an exact posterior by enumeration.
- 03 — Multiply factors and sum out a variable.
- 04 — Trace variable elimination step by step.
- 05 — Explain why elimination order controls cost.
- 06 — Compare rejection sampling and likelihood weighting.

Inference is organized marginalization

All methods account for hidden variables; they differ in when terms are reused or approximated.

- Enumeration expands every hidden assignment.
- Elimination distributes sums through products.
- Sampling replaces an exact sum with an empirical estimate.

Reuse

Variable elimination caches algebra in intermediate factors instead of recomputing identical sub-sums.

The posterior query

$$P(Q|e) = \alpha \sum_h \prod_i P(x_i | \text{parents}_i)$$

- Q denotes query variables.
- e fixes evidence variables.
- h ranges over hidden assignments.
- α normalizes entries to sum to one.

ENGINEERING IMPLICATION

Evidence is fixed before hidden variables are marginalized.

Factors are small functions, not full distributions

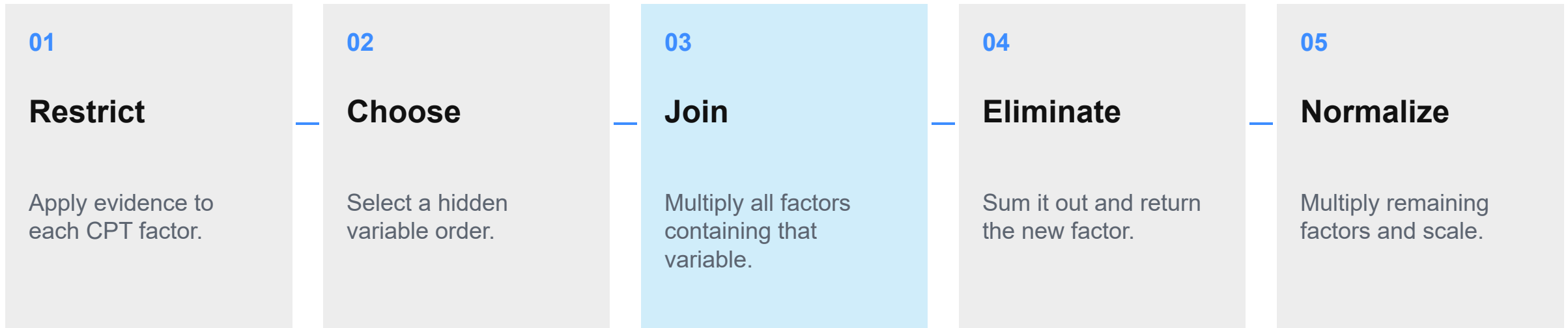
Factor multiplication

- Join factors on shared variables.
- Multiply compatible entries.
- Scope is the union.
- Order affects intermediate size.

Sum out

- Choose one hidden variable.
- Sum entries over its values.
- Remove it from scope.
- Preserve remaining function values.

Variable elimination reorganizes exact inference



Eliminate a variable only after collecting every factor that mentions it.

Variable-elimination skeleton

```
factors = restrict(CPT_factors, evidence)
for z in elimination_order(hidden):
    related = [f for f in factors if z in scope(f)]
    factors.remove(related)
    joined = pointwise_product(related)
    factors.add(sum_out(z, joined))
result = pointwise_product(factors)
return normalize(result over query)
```

Implementation contract

- Track factor scopes.
- Never sum evidence.
- Use stable variable order.
- Normalize last.

Enumeration on a two-cause alarm

SCENARIO

B and E are independent roots; A depends on both. We observe A=true and query B.

- For B=true, sum $P(B)P(E)P(A|B,E)$ over E.
- For B=false, repeat the same sum over E.
- This gives two unnormalized masses.
- Divide each by their sum.

The normalized pair is $P(B|A)$; enumeration recomputes the E contribution separately for each B value.

Exact methods trade simplicity for reuse

Enumeration

Expand assignments

- Conceptually direct.
- Repeated subexpressions.
- Exponential hidden settings.
- Good hand oracle.

Elimination

Reuse factors

- Exact result.
- Order-dependent cost.
- Intermediate tables.
- Good sparse networks.

Compilation

Many queries

- Reuse structure further.
- Up-front cost.
- Memory tradeoff.
- Beyond this lesson.

Checkpoint: factor scope

CHECKPOINT

Factors $f(A,B)$ and $g(B,C)$ are multiplied and B is summed out. What is the final scope?

01

Take the union first.

02

Remove only the eliminated variable.

03

Name variables still needed.

Evidence rarity separates sampling methods

Rejection sampling

- Sample every node ancestrally.
- Discard evidence mismatches.
- Unweighted surviving counts.
- Wasteful for rare evidence.

Likelihood weighting

- Clamp evidence values.
- Sample non-evidence nodes.
- Multiply evidence likelihoods.
- Can suffer weight degeneracy.

Likelihood-weighted estimate

$$w(\text{sample}) = \prod_{E_i \text{ in evidence}} P(e_i \mid \text{parents}(E_i)); \hat{P}(Q=q|e) \propto \sum w \cdot 1[Q=q]$$

- Evidence variables are not sampled.
- Each evidence CPT contributes a weight.
- Weighted query counts are accumulated.
- Final counts are normalized.

ENGINEERING IMPLICATION

No sample is rejected, but a few extreme weights can dominate.

Elimination reuses an upstream sum

SCENARIO

In $B \rightarrow A \leftarrow E$ with observed A , eliminate E before answering the two B values.

- Restrict A 's factor to $A=\text{true}$.
- Multiply $P(E)$ with $P(A=\text{true}|B,E)$.
- Sum E once to create $\phi(B)$.
- Multiply $\phi(B)$ by $P(B)$ and normalize.

The intermediate factor $\phi(B)$ stores the repeated E marginal needed by both posterior entries.

Elimination orders control the largest factor

Leaf first

Often local

- Small scopes.
- Early simplification.
- Good heuristic.
- Not always optimal.

Hub first

Often costly

- Joins many neighbors.
- Creates broad factor.
- Memory spikes.
- Exposes dense structure.

Heuristic order

Min-fill idea

- Estimate added connections.
- Choose low-growth variable.
- Deterministic ties.
- Record actual max scope.

Checkpoint: choose an inference method

CHECKPOINT

Evidence has probability one in a million and an exact factor would contain 2^{30} entries. Which introductory sampler is preferable and why?

01

Estimate rejection efficiency.

02

Consider weight variance.

03

State that approximation has uncertainty.

Test inference against normalization identities

Inference code should expose factor scopes, operation counts, and independent probability checks.

- Verify every returned distribution sums to one.
- Compare elimination with enumeration on random small DAGs.
- Report sample count, seed, effective sample size, and error.

Oracle

A tiny network can be enumerated completely and serve as a ground truth for elimination and sampling.

Key takeaways

- 01 — Enumeration is exact but repeats work.
- 02 — Variable elimination reuses intermediate factors.
- 03 — Order controls the largest intermediate scope.
- 04 — Sampling trades exactness for scalable estimates.

Next: Module C begins with supervised learning and linear regression from first principles.

Check your understanding

Q1

Write a posterior as a normalized sum.

Q2

Multiply two factor scopes.

Q3

Trace one elimination.

Q4

Explain a poor order.

Q5

Compare two samplers.